SHERLOCK

# Security Review For World Markets



Collaborative Audit Prepared For: **World Markets**
Lead Security Expert(s): [redacted]
Date Audited: **October 28 - December 9, 2025**

# Introduction

World Markets is the canonical DEX on MegaETH. It is the first trading venue to combine spot, under-collateralized lending, and perpetual futures under one portfolio-level risk engine. Your vault investments, unrealized PnL, staked assets, and loans all contribute to your available margin to trade.

## Scope

Audited Commit: 36ece93f76fc324515e5c9666523e22f607cbffe

Final Commit: a3fd22029ebbdd1ac830d1ad3bc0ab32cb015894

## Findings

**All issues have been addressed, resolved, and their fixes verified by the team of Security Researchers.**

## Issues Found

| High | Medium | Low/Info |
|---|---|---|
| 6 | 12 | 5 |

Each issue has an assigned severity:

- High issues are directly exploitable security vulnerabilities that need to be fixed.
- Medium issues are security vulnerabilities that may not be directly exploitable or may require certain conditions in order to be exploited. All major issues should be addressed.
- Low/Info issues are non-exploitable, informational findings that do not pose a security risk or impact the system's integrity. These issues are typically cosmetic or related to compliance requirements, and are not considered a priority for remediation.

# Issue H-1: Unexpected cancellation leads to loss of funds [RESOLVED]

## Summary

In the spot/perp order book, it incorrectly calculates the balance of sell node.

## Vulnerability Detail

In the `checkSellBalance` function, it calculates the balance of the sell node. When it calculates the balance of the node, it should divide by `config.fromVaultPdDiff()` rather than `toVaultPdDiff()`.

```
(uint128 bal,, ) = vault.getBalance(sellNodeData.client(), config.fromTokenId());
uint64 maxQuantity = uint64(bal / 10**(config.toVaultPdDiff()));
if (maxQuantity == 0) {
    return 0;
}
```

## Impact

- User orders are cancelled unexpectedly.
- In the spot market, it affects user's position and users can't close their position as they want. It leads to loss of funds to the users.
- DoS of filling order.

## Tool Used

Manual Review

## Recommendation

```
        (uint128 bal,, ) = vault.getBalance(sellNodeData.client(),
        ↪  config.fromTokenId());
-       uint64 maxQuantity = uint64(bal / 10**(config.toVaultPdDiff()));
+       uint64 maxQuantity = uint64(bal / 10**(config.fromVaultPdDiff()));
```

## Discussion

**moh-eulith**

2nd point under ”Impact” is incorrect. This code only applies to Spot.

**moh-eulith**

Commit 3842ffc

# Issue H-2: Utilizing set Instead of increase [RESOLVED]

## Vulnerability Detail

'set' operation is used instead of an 'increase'. As a result, `lendHoursDuration` cannot exceed 10 days. Additionally, since `hoursPaid` can be greater than `lendHoursDuration`, this may lead to an underflow, resulting in a DoS and causing several functions, including liquidation, to fail.

```
56:             if (extendPeriod) {
57:                 lendMatch = lendMatch.setLendHoursDuration(240 -
 ↪  (lendMatch.lendHoursDuration() - lendMatch.hoursPaid()));
58:             }
```

## Impact

The core functionality of the contract is broken: the borrow period cannot be extended. This issue may lead to a loss of funds, as liquidation might not work for some users, potentially resulting in bad debt.

## Tool Used

Manual Review

## Recommendation

```
-       lendMatch = lendMatch.setLendHoursDuration(240 -
 ↪  (lendMatch.lendHoursDuration() - lendMatch.hoursPaid()));
+       lendMatch = lendMatch.setLendHoursDuration(240 + lendMatch.hoursPaid());
```

## Discussion

**moh-eulith**

Commit a5c3c21

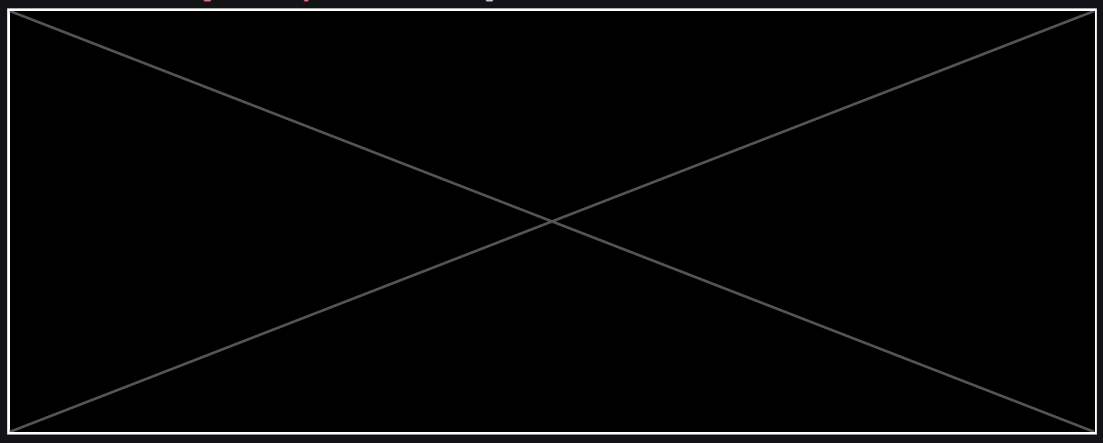
# Issue H-3: Original interest rate is passed on to the borrower [RESOLVED]

## Vulnerability Detail

In the doc:

> If the interest rate is lower than the position they had, the lower interest rate is passed to the borrower. However, in this case, the original interest rate is passed on to the borrower.

```
662:     function lenderReplacedMatch(LendMatch existing, LendMatch newLender,
 ↪  uint64 _quantity) internal pure returns(LendMatch) {
```



```
675:         return LendMatch.wrap(v);
676:     }
```

## Impact

Borrowers pay more interest than they should. Lenders receive more interest than their request.

## Discussion

**moh-eulith**

Commit a5c3c21

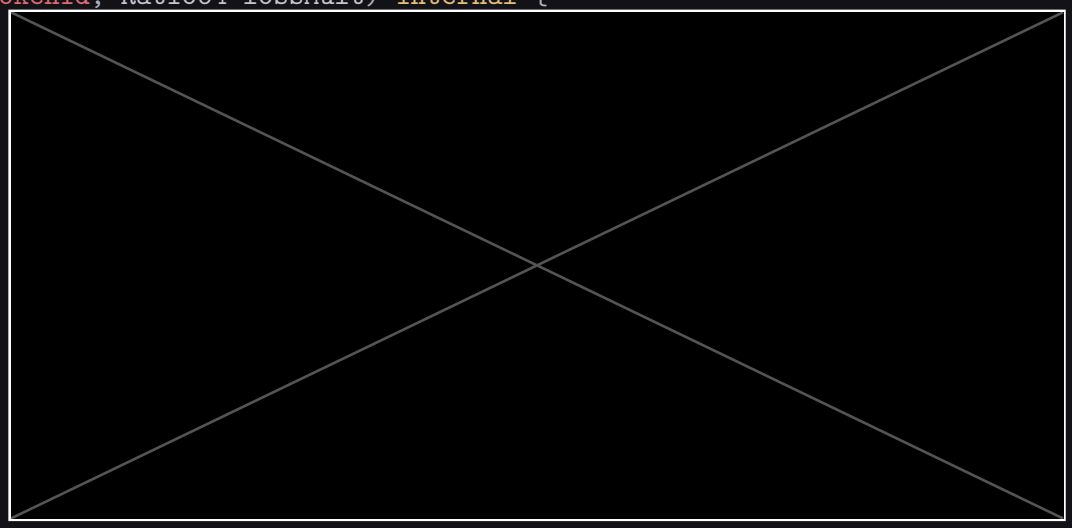
# Issue H-4: borrowerQuantity and spotBalance [RESOLVED]

## Vulnerability Detail

The [redacted] comparison is incorrect. Due to this, [redacted]ExchangeExt.sol#L258 could be underflow. And [redacted]ExchangeExt.sol#L314 can be skiped.

```
323:     function processPayableBorrowsAtLossSellExtraForToken(uint64 userId,
 ↪  uint32 tokenId, Ratio64 lossMult) internal {
324:
325:
326:
327:
 ↪  tokenCon
328:

333:
334:
335:
336:
337:
338:
339:
           ...
```



```
252:         LendAggPosition agg = readLendAgg(userId, tokenId);
253:         if (agg.borrowerQuantity() > 0) {
254:             VaultTokenConfig tokenConfig = internalReadTokenConfig(tokenId);
255:             VaultTokenConfig baseConfig =
 ↪  internalReadTokenConfig(BASE_TOKEN_ID);
256:             VaultLedger ledger = internalReadLedger(userId, tokenId);
257:             uint64 spot =
 ↪  tokenConfig.convertVaultToPosition(ledger.userBalance());
258:             [redacted];
```

## Impact

A malicious user can make a address which can't be liquidated.

## Discussion

**moh-eulith**

Commit f375cb5

# Issue H-5: internalPayInterestAndFees() may revert unreasonably. [RESOLVED]

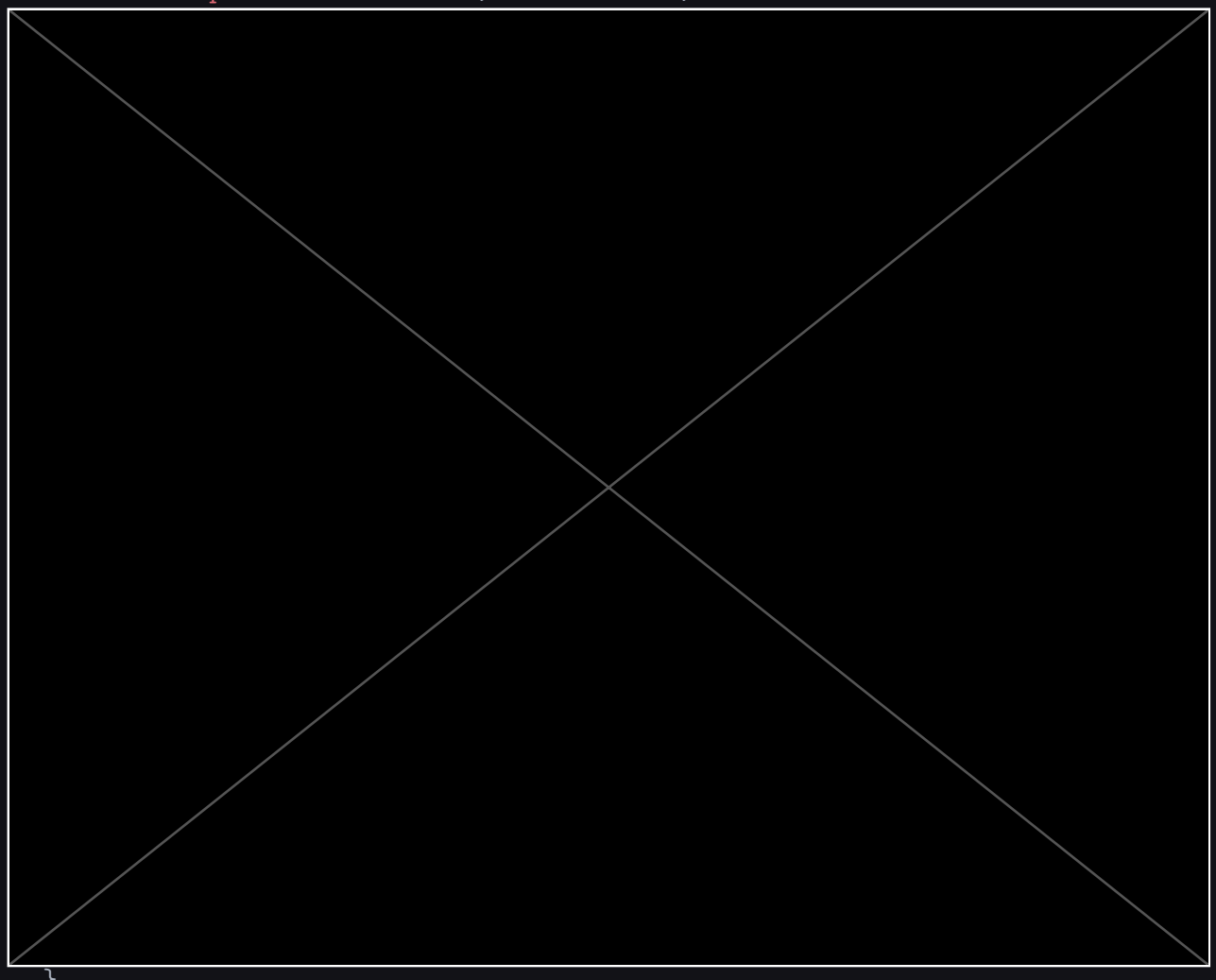
## Summary

When paying interest and principle, new lend order can be be created. So, paying can revert if the lender has no enough balance.

### Vulnerability Detail

The application of the sequestrationMultiplier is not suitable. If a lender consumes all of its maxSequesters and then matches some of them, the maxSequesters decreases significantly by (delta * sequestrationMultiplier), while the spotLendSeqBalance decreases only slightly by (delta). This can lead to exceeding the limit and creating a vulnerability.

```
function sequester(uint64 user, uint32 token, uint64 amount) external
```



```
}
```

`internalPayInterestAndFees()` can create a new lend order when reducing borrowed amount.

```
        if (lendMatch.returnLenderToBookAsBit() == 1 && reduceQuantity > 0) {
            address orderBook = internalGetLendBook(tokenId);
            ILendBookForExch book = ILendBookForExch(orderBook);
            LendOrder orderData =
             ↪  LendOrderLib.newLendOrder(lendMatch.lenderAccountId(),
             ↪  reduceQuantity, lendMatch.lenderInterestRate());
@>          book.newLendSellOrder(orderData);
        }
```

However, the `newLendSellOrder()` function can revert due to some reasons, for example, in the `sequester()` function.

Consider the following scenario:

1. Alice's balance is 100.
2. Alice made 20(=sequestrationMultiplier) lend orders whose amounts are 100. So, Alice's spotLendSeqBalance is 2000.
3. Two lend orders are matched by Bob and Charlie. Each of their amounts are 50. So, Alice's balance is 0. Alice's spotLendSeqBalance is 1900.
4. Bob is going to pay 50.

In the above scenario, Bob's pay will fail because: 1900 + 50 > 50 * 20

## Impact

Users may be unable to pay for their positions.

## Tool Used

Manual Review

## Recommendation

try/catch mechanism should be used for creating new lend order.

## Discussion

**moh-eulith**

Commit 5355ecb

# Issue H-6: A malicious user can avoid intereset payment by using lifoLenderSwap [RESOLVED]

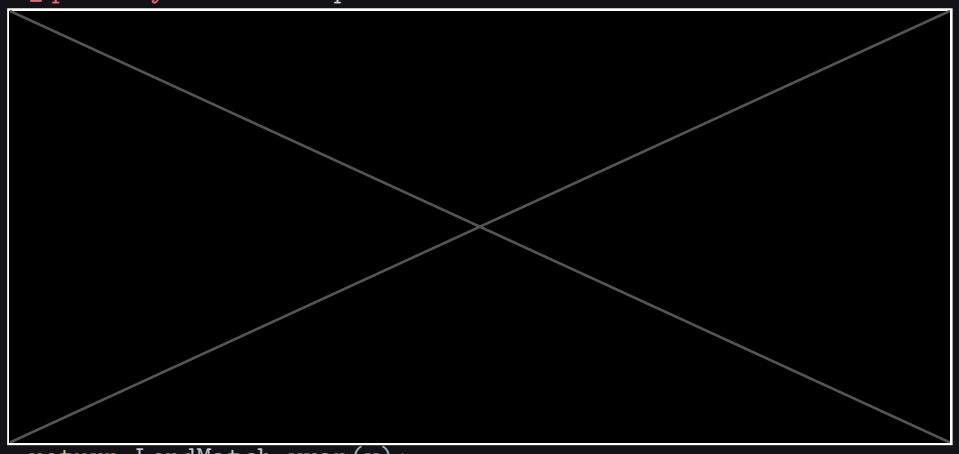
## Summary

During the swapLenders process, the pending interest in both the toSwapBorrowed position and the existingLent position is not settled. The toSwapBorrowed position is replaced with the startTime and hoursPaid of the existingLent position.

In other words, the swapper does not receive the interest they should receive and does not pay the interest they should pay. As a result, swapper can exploit this vulnerability to avoid paying borrowing interest of old position.

## Vulnerability Detail

`lifoLenderSwap` is used to cancel lending and borrowing at the same time. However, the new position's `hoursPaid` is set as the original `hoursPaid`. So, a malicious user can avoid intereset payment.

```
    function lenderReplacedMatch(LendMatch existing, LendMatch newLender, uint64
    ↪  _quantity) internal pure returns(LendMatch) {
@>
    
        return LendMatch.wrap(v);
    }
```

Consider the following scenario:

1. Alice borrows 100 from Bob and lends 100 to Charlie.
2. Charlie pays interest.
3. Alice cancels lending and borrowing at the same time.
4. Alice earns interest for free.

## Impact

A malicious user can avoid intereset payment by using `lifoLenderSwap`.

## Tool Used

Manual Review

## Recommendation

The new position should be made using considering both of original positions.

## Discussion

**moh-eulith**

Commit 29c1ee4 and typo fixed in Commit a799a5b

(it's no longer possible to swap in old positions)

# Issue M-1: setMinOrderQuantity function removes fee information [RESOLVED]

## Summary

Whenever the `setMinOrderQuantity` is called, fee schedule information is removed.

## Vulnerability Detail

For a lend orderbook config, the information of the token ID is stored in the `config.toTokenID()`.

```
function newConfigForLending(uint32 tokenId) internal pure returns(OrderBookConfig)
 ↪  {
    [redacted]
```

So, `config.fromTokenID()` will always return `0` for the lend orderbook type.

```
function fromTokenId(OrderBookConfig obc) internal pure returns (uint32) {
    return uint32(OrderBookConfig.[redacted]);
}
```

As a result, whenever the admin calls the `setMinOrderQuantity` function, fee schedule information is removed.

## Impact

Loss of fees.

## Tool Used

Manual Review

## Recommendation

```
-       LendFeeSchedule fees = internalGetLendFeeSchedule(config.fromTokenId());
+       LendFeeSchedule fees = internalGetLendFeeSchedule(config.toTokenId());
```

## Discussion

**moh-eulith**

Commit 3842ffc

# Issue M-2: When it cancels order, it does not write updated list meta data [RESOLVED]

## Summary

When it cancels order, it does not update the list meta data.

## Vulnerability Detail

In the `internalCancelSpotBuyOrder` and `internalCancelSpotSellOrder` functions, it first removes the node from corresponding area. However, it does not write the updated information to the storage.

## Impact

It is impossible to create orders since the length of list never decreases.

## Tool Used

Manual Review

## Recommendation

```
         // free the node:
-        updateListMetaDataForFreedNode(ListMetaLib.read(area), area, pos);
+        ListMeta listMeta = updateListMetaDataForFreedNode(ListMetaLib.read(area),
 ↪  area, pos);
+        StorageUtils.writeStorage(area, 0, listMeta.raw());
     }
```

## Discussion

**moh-eulith**

Commit 3842ffc

# Issue M-3: Reconfiguration does not update the minOrderQuantity of the order book [RESOLVED]

## Summary

When it reconfigures the setting of lend order book, it does not sync with the parameter of the order book.

## Vulnerability Detail

In the `reconfigureLendBook` , it can set a new `minOrderQuantity` of the order book config. However, it does not update the parameter of the order book. It can use the staled parameter.

## Impact

The `minOrderQuantity` parameter of the order book is not synced with the main config.

## Tool Used

Manual Review

## Recommendation

```
    function reconfigureLendBook(address orderBook, LendFeeSchedule fees) external
     ↪  onlyAdmin {
        OrderBookConfig old = OrderBookConfig.wrap(StorageUtils64.readStorageForAd
         ↪  dress(AREA_ORDERBOOK_TO_CONFIG, orderBook));
        require(!old.isBlank() && old.bookType() == LEND_BOOK_TYPE,
         ↪  ExchangeErrors.NoOrderBookRegistered());
+       fees = setMinOrderQuantityFromBook(orderBook, fees);
        StorageUtils64.writeStorageForAddress(AREA_LENDBOOK_TO_FEE_SCHEDULE,
         ↪  orderBook, fees.raw());
    }
```

## Discussion

**moh-eulith**

Commit 3842ffc

# Issue M-4: DoS of swapByAmountInViaMinOut function [RESOLVED]

## Summary

In the `computeBuyMarketOrderSpecifyIn` and `computeSellMarketOrderSpecifyIn` functions, it does not update node data for the head.

## Vulnerability Detail

In the `computeBuyMarketOrderSpecifyIn` and `computeSellMarketOrderSpecifyIn` functions, it updates the `head`, but does not udpate `sellNodeData` and `buyNodeData`. It results incorrect estimated price for Swap Router.

## Impact

DoS of the `swapByAmountInViaMinOut` function for Buy/Sell operation.

## Tool Used

Manual Review

## Recommendation

```
            head = buyNodeData.next();
+           if (head != 0) {
+               buyNodeData = SpotNodeLib.read(BUY_ORDER_AREA, head);
+           }

            head = sellNodeData.next(); // head = head.next
+           if (head != 0) {
+               sellNodeData = SpotNodeLib.read(SELL_ORDER_AREA, head);
+           }
```

## Discussion

**moh-eulith**

Commit b434884

# Issue M-5: DoS of swapByAmountOutViaMaxIn function [RESOLVED]

## Summary

In the `computeBuyMarketOrderSpecifyOut` and `computeSellMarketOrderSpecifyOut` functions, it does not update node data for the head.

## Vulnerability Detail

In the `computeBuyMarketOrderSpecifyOut` and `computeSellMarketOrderSpecifyOut` functions, it updates the `head`, but does not udpate `sellNodeData` and `buyNodeData`. It results incorrect estimated price for Swap Router.

## Impact

DoS of the `swapByAmountOutViaMaxIn` function for Buy/Sell operation.

## Tool Used

Manual Review

## Recommendation

```
            head = buyNodeData.next();
+           if (head != 0) {
+               buyNodeData = SpotNodeLib.read(BUY_ORDER_AREA, head);
+           }

            head = sellNodeData.next(); // head = head.next
+           if (head != 0) {
+               sellNodeData = SpotNodeLib.read(SELL_ORDER_AREA, head);
+           }
```

## Discussion

**moh-eulith**

Commit b434884

# Issue M-6: No overflow check in the times10 function [RESOLVED]

## Summary

There is no overflow check in the `times10` function.

## Vulnerability Detail

In the `times10` function, there is no overflow check. The expression `(p.mantissa() * 10) << 5` could potentially overflow a uint64. If an overflow occurs, the mantissa will be close to zero.

## Impact

In the swap router, buy operations via the `swapByAmountOutViaMaxIn()` and `swapByAmountInViaMinOut()` functions will be DoSed.

## Tool Used

Manual Review

## Recommendation

```
        if (exp == 0) {
-           return Price59EN5.wrap(uint64((p.mantissa() * 10) << 5));
+           uint v = p.mantissa() * 10;
+           require(v < (1 << 59));
+           return Price59EN5.wrap(uint64(v) << 5);
        }
```

## Discussion

**moh-eulith**

Commit 3842ffc

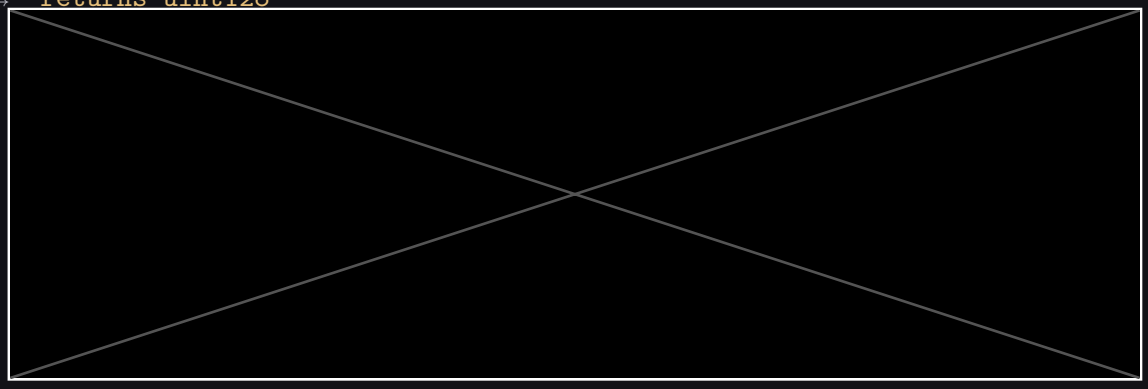
# Issue M-7: Funding Rate calculation can be incorrect [RESOLVED]

## Summary

Funding rate calculation can be incorrect.

## Vulnerability Detail

In the `divScaled30` function, the ratio can overflow the `uint128.max` and it could return smaller value.

```
    function divScaled30(Price59EN5 numer, Price59EN5 denom) internal pure
    ↪  returns uint128
    {
@>      
    }
```

## Impact

It results in incorrect funding rate calculation and accouting.

## Tool Used

Manual Review

## Recommendation

Add overflow check.

## Discussion

**moh-eulith**

Commit a3fd220

# Issue M-8: With sufficient balance, users can't borrow due to overflow [RESOLVED]

## Summary

A user with large balance can't borrow in the lend market.

## Vulnerability Detail

In the `invertedBuySequesterAmount` function, it calculates the sequester amount and casts it to `uint64`.

```
    function invertedBuySequesterAmount(uint64 availableInterest, uint16
    ↪  _interestRate) internal pure returns (uint64) {
@>      return uint64((availableInterest) *
↪   (365*3*ConfigConstants.INTEREST_RATE_DIVISOR) / w16(_interestRate));
    }
```

As a result, it returns smaller amount than expected.

## Impact

Users with large balance can't borrow effectively in the lend market.

## Tool Used

Manual Review

## Recommendation

Add overflow check.

## Discussion

**moh-eulith**

Commit a5c3c21

# Issue M-9: Incorrect cast of user balance [RESOLVED]

## Summary

In the lend market, it incorrectly casts user balance to `uint64`.

## Vulnerability Detail

In the `checkBuyBalance` and `checkSellBalance` functions, it first gets a user balance from the ledger. Then, it casts it to `uint64`. If a user has a large balance, it returns the small value.

```
        (uint128 bal,, ) = vault.getBalance(_buyIterator.userId(),
        ↪  tokenConfig.tokenId());
@>      uint64 positionBal = uint64(bal / 10**(tokenConfig.vaultPdDiff()));
```

## Impact

Users with large balance can't borrow effectively in the lend market.

## Tool Used

Manual Review

## Recommendation

Add overflow check.

## Discussion

**moh-eulith**

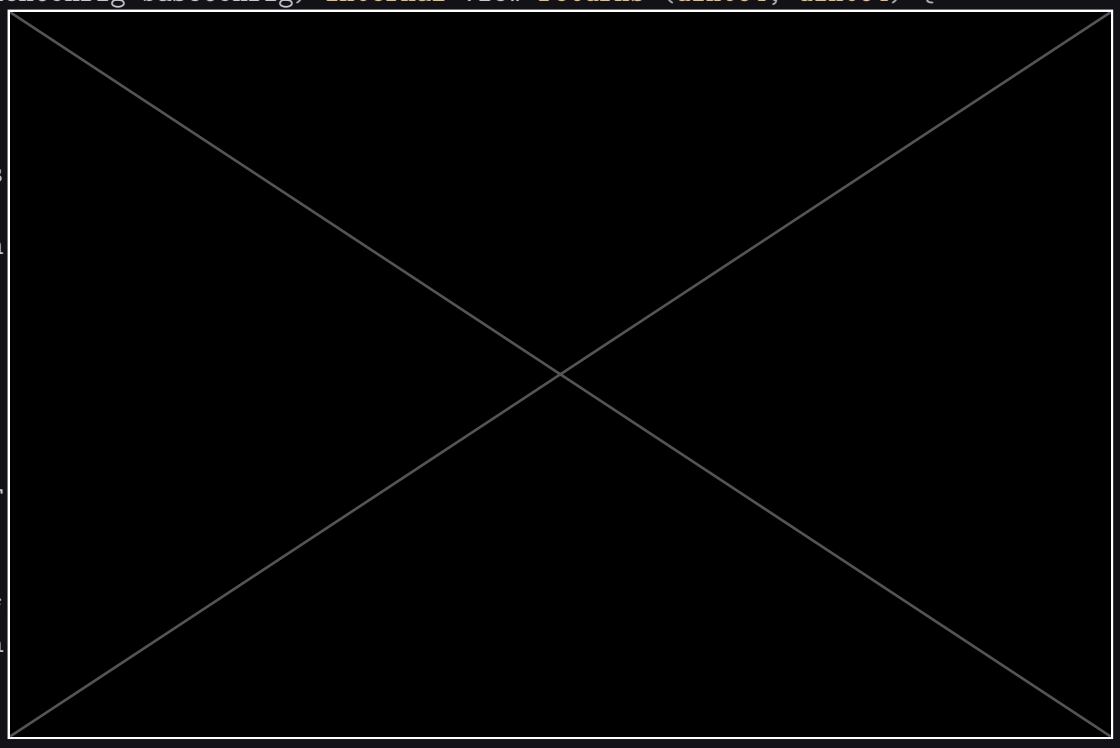
Commit a5c3c21

# Issue M-10: The Credit of lossMult Does Not Include the Lending Position [RESOLVED]

## Vulnerability Detail

lossMult consists of both credits and debts. However, credits do not include the liquidatee's lending positions. If there are no lending orders in the order book, undoNonBaseLenderPositions may not completely remove the lender position. In a bankruptcy scenario, all debts can be liquidated by applying lossMult, which leaves the liquidatee with no debts and only lending positions remaining.

```
352:     function computeCreditLiability(uint64 userId, uint32 tokenId,
 ↪  VaultTokenConfig baseConfig) internal view returns (uint64, uint64) {
353:
354:
355:
356:
 ↪  int8(bas
357:
 ↪  tokenCon
358:
359:
360:
361:
362:
363:
 ↪  convertF
364:
365:
 ↪  mark.inf
 ↪  - spotBa
366:
367:
368:     }
```



## Impact

This situation could lead to potential losses for the lenders and profits for the liquidatee.

## Recommendation

If there are any outstanding lending positions, I believe they should be distributed appropriately among the lenders. Alternatively, I think the admin should add a lending order to prevent this situation.

## Discussion

**moh-eulith**

Commit a3fd220

# Issue M-11: A lending position with the same borrower and lender can be created by lifoLenderSwap() [RESOLVED]

## Summary

The `lifoLenderSwap()` function is used to decrease the lend amount and borrow amount for a user and a token. At that time, a position with the same lender and borrower can be made.

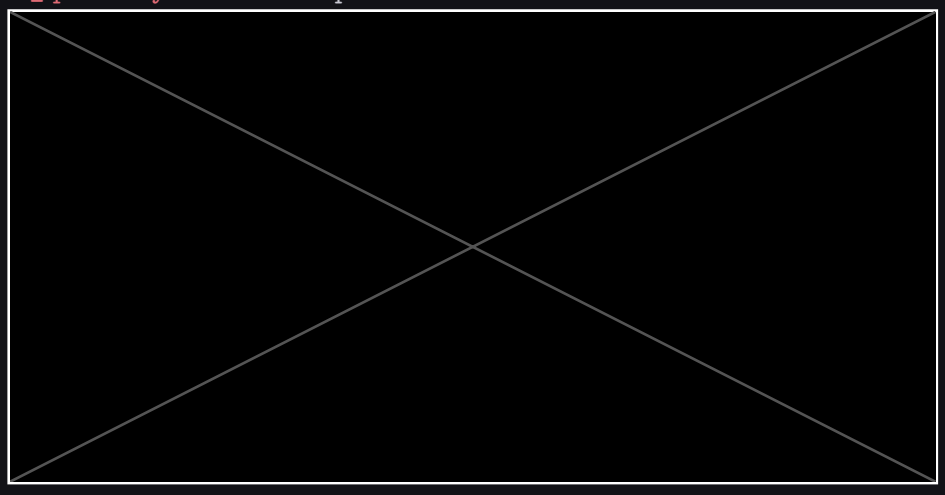
## Vulnerability Detail

The `lifoLenderSwap()` function is used to decrease the lend amount and borrow amount for a user and a token.

```
function lifoLenderSwap(uint64 lentPositionId) external {
    LendMatch lendMatch = readLendMatch(lentPositionId);
    checkTradingPermission(lendMatch.lenderAccountId());
    internalLifoLenderSwap(lendMatch, lentPositionId);
}
```

There is no check if the borrower and the lender is the same.

```
function lenderReplacedMatch(LendMatch existing, LendMatch newLender, uint64
↪  _quantity) internal pure returns(LendMatch) {
```



```
}
```

Consider the following scenario: initial state: position 1 (amount: 100, lender: Alice, Borrower: Bob) position 2 (amount: 100, lender: Bob, Borrower: Alice) If Alice calls `lifoLenderSwap()` for position 1, a new position with Bob as the Borrower and lender can be made.

## Impact

A lendMatch with the same lender and borrower can be made.

## Tool Used

Manual Review

## Recommendation

In the `lifoLenderSwap()`, the above case should be prvented.

## Discussion

**moh-eulith**

Commit a3fd220

**moh-eulith**

I consider this ”Medium” because a self-lend would cause issues with liquidation.

# Issue M-12: A malicious user can cause lending position key collision. [RESOLVED]

## Summary

The key(64bit) of a new lending position is made by combining `nextLendingSequence()` (32bit) and `nowMinutes()`(32bit). However, the first 32 bit can be reset when swapping lenders. So, there is a risk of position key collision.

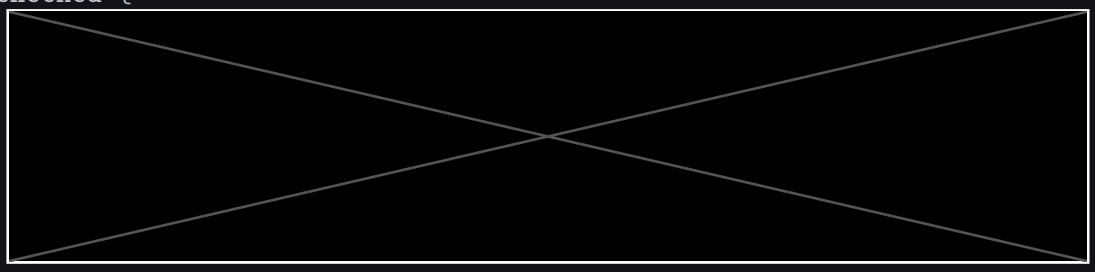
## Vulnerability Detail

The key(64bit) of a new lending position is made by combining `nextLendingSequence()` (32bit) and `nowMinutes()`(32bit).



`nextLendingSequence()` returns values between 1 and 2^30.

```
function nextLendingSequence() internal returns (uint32) {
    unchecked {
        
        
        
        
    }
}
```



However, the first 32 bit can be reset when swapping lenders.

```
uint64 newMatchId = (uint64(nextLendingSequence()) << 32) |
↪   uint64(uint32(existingLentId));
```

Consider the following scenario:

1. Two positions are made in the same minute. Position 1: `nextLendingSequence()` = 1, `nowMinutes()` : 2000 Position 2: `nextLendingSequence()` = 2, `nowMinutes()` : 2000
2. After some time, the position 1 still exists and the key of position 2 is reset as 1 when swapping lenders.

Then the position 1 will be erased, which can result in very severe loss of fund for users. In step 2, a malicious user can cause the collision.

## Impact

Erase of a lending position.

## Tool Used

Manual Review

## Recommendation

If collision occurs, the number should be skipped.

## Discussion

**moh-eulith**

Commit 29c1ee4

Disagree with "Medium". See discussion in PR.

 **dmin**

I agree that an attack is currently impossible, and I also agree that the probability of an accident occurring is extremely low. However, I do not believe we can definitively state that an accident is impossible. If the lending market becomes very active and nextLendingSequence reaches 1 after 7 years, the possibility of an attack or accident could arise, and the potential impact would be high. According to the Sherlock guide, I think it's suitable for Medium.

> Note Likelihood is not considered when identifying the severity and the validity of the report.

# Issue L-1: Incorrect scaling in the divide10 function [RESOLVED]

## Summary

Incorrect scaling in the `Price59EN5.divide10` function.

## Vulnerability Detail

In the `divide10` function, it divides the value by 100 rather than by 10.

## Impact

It returns the lower value than expected.

## Tool Used

Manual Review

## Recommendation

```
-            return newPrice59EN5(uint64(p.mantissa()/10), uint8(p.unsignedExp() +
 ↪  1));
+            return newPrice59EN5(uint64(p.mantissa()), uint8(p.unsignedExp() + 1));
```

## Discussion

**moh-eulith**

Commit 3842ffc

# Issue L-2: dust lend position [RESOLVED]

## Vulnerability Detail

In the line 216, when quantity() == minOrderQuantity, the loop is skiped.

```
206:     function internalLifoLenderSwap(LendMatch lendMatch, uint64
 ↪  lentPositionId) internal {
207:         LendMatch original = lendMatch;
208:         uint64 borrowerArea =
 ↪  userIdToLendingLists(lendMatch.lenderAccountId());
209:         uint64 borrowListId = tokenIdToBorrowerListId(lendMatch.tokenId());
210:         if (StorageUtils64.isListEmpty(borrowerArea, borrowListId)) {
211:             return;
212:         }
213:         (LifoIterator it, uint64 lastItem) =
 ↪  StorageUtils64.lifoLoop(borrowerArea, borrowListId);
214:         LendFeeSchedule feeSched =
 ↪  internalGetLendFeeSchedule(lendMatch.tokenId());
215:         lendMatch = swapLenders(lendMatch, lentPositionId, lastItem, feeSched);
216:         while (lendMatch.quantity() > feeSched.minOrderQuantity() &&
 ↪  it.hasNext()) {
217:             (it, lastItem) = it.loopNext(borrowerArea, borrowListId);
218:             lendMatch = swapLenders(lendMatch, lentPositionId, lastItem,
 ↪  feeSched);
219:         }
220:         if (StorageUtils64.isListEmpty(borrowerArea, borrowListId)) {
221:             checkPerpDebtBitmapForReset(lendMatch.lenderAccountId(),
 ↪  lendMatch.tokenId());
222:         }
223:         LendAggPosition p = readLendAgg(lendMatch.lenderAccountId(),
 ↪  lendMatch.tokenId()).decrementLenderQuantity(original.quantity() -
 ↪  lendMatch.quantity());
224:         p = p.decrementBorrowerQuantity(original.quantity() -
 ↪  lendMatch.quantity());
225:         writeLendAgg(lendMatch.lenderAccountId(), lendMatch.tokenId(), p);
226:     }
```

## Impact

In the liquidation process, their could be remain lend position which has minOrderQuantity.

## Discussion

**moh-eulith**

Commit a5c3c21

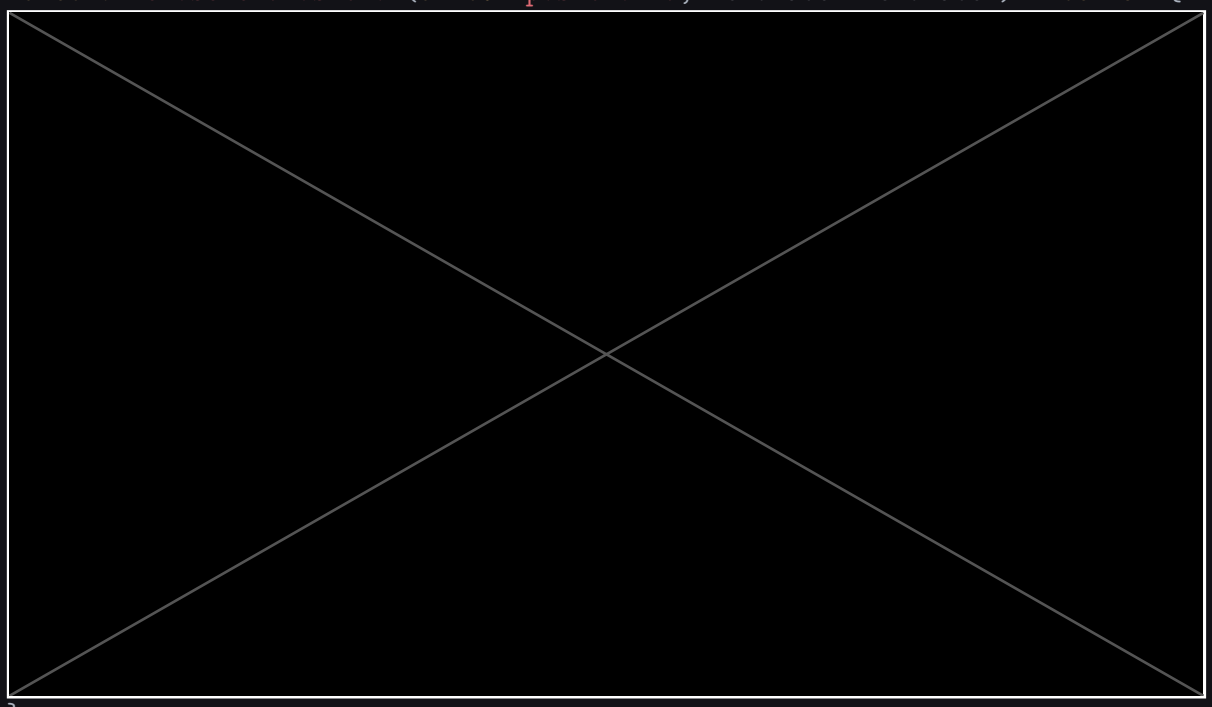
# Issue L-3: noDebt bit should be reset in the closeLendPosition() [RESOLVED]

## Summary

In the `closeLendPosition()` function, `noDebt` bit is not reset because the lender is guaranteed to have non-zero token balance. However, in some cases, the lender may have zero balance.

## Vulnerability Detail

In the `closeLendPosition()` function, `noDebt` bit is not reset because the lender is guaranteed to have non-zero token balance.

```
    function closeLendPosition(uint64 positionId, LendMatch lendMatch) internal {
@>  
    }
```

However, in some cases, the lender may have zero balance. For example, if the position has the same borrower and lender, the position may have zero balance. As a result, `noDebt` bit is incorrectly set for an account, which can result in critical issue because the bit is used in calculating portfolio of the account. Malicious users deliberately cause this to happen.

## Impact

`noDebt` bit is incorrectly set for an account.

## Tool Used

Manual Review

## Recommendation

`noDebt` bit should be checked in the `closeLendPosition()`.

## Discussion

**moh-eulith**

As per #50 self-lend is no longer possible.

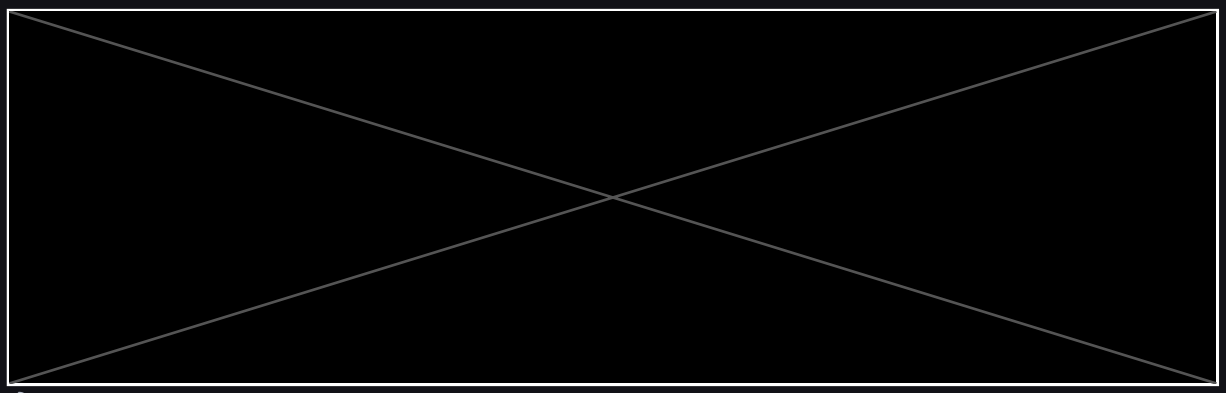
# Issue L-4: clienId is unnecessary [RESOLVED]

## Summary

The `clientId` is unnecessarily included in the `or` operation twice.

## Vulnerability Detail

In the `TwoTokenOrderBook.internalNewSpotBuyOrder()` function, `clientId` is included in the `orderData.clientQuantityPrice()`, but both participate in the `or` operation. It is acceptable to remove `clientId` from this operation.

```
    function internalNewSpotBuyOrder(SpotOrder orderData) internal returns
    ↪  (BuyOrderResult){
        ...
@>
↪   
    }
```

## Tool Used

Manual Review

## Recommendation

```
    function internalNewSpotBuyOrder(SpotOrder orderData) internal returns
    ↪  (BuyOrderResult){
        ...

-       SpotNode nodeData = SpotNode.wrap(                              ) |
↪   (                ) | orderData.clientQuantityPrice());
+       SpotNode nodeData = SpotNode.wrap(                              ) |
↪   orderData.clientQuantityPrice());
        //nodeData = prev 32 | next 32 | order seq 20 | clientId 44 | sellQuantity
        ↪  64 | sellPrice 64
        if (pos != 0) {
```

```
            insertIntoBuyListFrom(pos, linkedListNodePos, nodeData, buyPrice);
        } else {
            insertIntoBuyList(linkedListNodePos, nodeData, buyPrice);
        }
        return result;
    }
```

## Discussion

**moh-eulith**

Commit 66f01df

# Issue L-5: There is no check for 0 quantity in perp order [RESOLVED]

## Summary

There is no check for 0 quantity in perp trading so it allows the 0 quantity perp order.

## Vulnerability Detail

In L26, it only checks quantity is the multiple of minOrderQuantity and doesn't check whether it is 0 amount.

```
File: contracts\src\main\sol\PerpOrderBook.sol
25:     function checkOrderQuantity(SpotOrder orderData) internal override view {
26:         require(orderData.quantity() % readMinOrderQuantity() == 0,
 ↪  ExchangeErrors.QuantityNotMultipleOfMin());
27:     }
```

## Tool Used

Manual Review

## Recommendation

Add check for quantity with 0.

```
    function checkOrderQuantity(SpotOrder orderData) internal override view {
        require(orderData.quantity() % readMinOrderQuantity() == 0,
         ↪  ExchangeErrors.QuantityNotMultipleOfMin());
+       require(orderData.quantity() > 0, "0 quantity is not allowed");
    }
```

## Discussion

**moh-eulith**

Commit 66f01df (also fixes the same issue in LendOrderBook)

**moh-eulith**

Note: the impact is not right. A zero quantity order cannot be created because it fails on an internal sequester check. The commit above simply changes the error returned.

# Disclaimers

Sherlock does not provide guarantees nor warranties relating to the security of the project.

Usage of all smart contract software is at the respective users' sole risk and is the users' responsibility.